Revised Complete Proto-Hive Migration Plan

Containerized deployment to ASI replacement VM1 and VM2

ZeroBot / ProtoCosmo migration workstream

2026-08-12

Decision. Docker Compose is the primary deployment path. The runtime, supervision, health checks, and receiver configuration live in reproducible containers. Persistent state and credentials remain in narrowly scoped host mounts. Cutover is performed one Telegram identity at a time, with exactly one active receiver and the Pop!_OS laptop retained as the immediate rollback host.

1 Purpose and current evidence

The objective is to migrate four proto-hive identities from the Pop!_OS laptop to two existing ASI replacement virtual machines without losing identity, routing, cursor state, scheduled work, or rollback capability. The target assignment is:

Target	Identities	Role
VM1	ProtoCosmo, ProtoCosmo2	Two isolated Compose services
VM2	Protomega, Protomega2	Two isolated Compose services

Observed before this revision:

- Both targets are reachable Ubuntu 22.04 hosts with 4 CPUs, about 16 GiB RAM, about 128 GB storage, and clean systemd state.
- Checksummed, credential-free source bundles have passed integrity, extraction, compile, and provider-free tests on both targets.
- Each target passed 113 provider-free tests; one laptop-specific PDF fixture test was deliberately excluded from remote staging.
- Fail-closed native staging units and immutable identity roots exist but remain disabled. No VM receiver has been enabled.
- A preliminary private ProtoCosmo2 state snapshot is staged on VM1 with polling disabled. The laptop remains authoritative for every receiver.

Containerization changes the packaging and supervision layer, not the safety contract. Existing verified source pins and state manifests are inputs to the image and mounted-volume design. The disabled native staging path remains a fallback until the containerized deployment passes acceptance.

2 Non-negotiable invariants

1. **Single receiver:** for each bot token, at most one process may poll Telegram at any instant. A target receiver starts only after its laptop owner has stopped and absence is independently verified.

2. **Per-identity isolation:** credentials, cursor/state, sessions, attachments, logs, and locks are never shared across identities.
3. **No secrets in artifacts:** image layers, Compose files, Git, project Markdown, logs, shell arguments, PDF, and chat contain credential categories or references only, never values.
4. **Fail closed:** absence of an explicit per-identity activation marker prevents polling. Health checks cannot enable a receiver.
5. **Immutable runtime:** source and dependencies are pinned and built into a versioned image. Runtime containers do not modify their code layer.
6. **Mutable state is durable:** every required mutable path is a bounded host bind mount or named volume with verified owner, mode, and backup.
7. **Rollback before optimization:** the laptop copy is preserved and restartable through the soak period. No cleanup precedes acceptance.
8. **One lane at a time:** only one identity enters the stop, final delta, start, canary, and adjudication sequence at once.
9. **Evidence before claims:** a lane is complete only when receipts, receiver counts, fresh canaries, restart checks, and state continuity evidence exist in the experiment ledger.

3 Target architecture

3.1 Portable container unit

Build one pinned base image containing the OpenClaw/Node runtime and common dependencies, plus pinned Omega, PeTTa, and per-identity runner inputs where needed. Select identity behavior through an explicit service command and read-only configuration, not through mutation of the image.

Each VM runs a project-scoped Docker Compose stack. Each identity is a separate service with:

- its own container name, internal network identity, health check, restart policy, resource limits, and log policy;
- read-only runtime/config mounts and separate read-write state mounts;
- a per-identity secrets directory mounted read-only;
- no published port unless a verified host integration requires one;
- polling disabled by default and gated by a host activation marker;
- a non-root runtime user where repository behavior permits it.

Docker Compose provides lifecycle orchestration. Host systemd owns one Compose-stack unit per VM so the stack starts after Docker and mounted storage, survives reboot, and has a single inspectable host-level supervisor. Container restart policy handles process failure; systemd handles Docker/host lifecycle.

3.2 Host/container boundary

Concern	In container/image	On host or mounted storage
Runtime	Pinned Node/OpenClaw, Python/PeTTa dependencies, reviewed runners	Docker Engine, Compose plugin, systemd stack unit
Identity config	Schema and non-secret defaults	Read-only identity-specific config and routing policy
Credentials	Names and loading mechanism only	Mode-restricted per-identity secret files or Docker secrets

State	State-handling code	Cursor, sessions, memory, schedules, attachment metadata, durable queues
Logs	Structured stdout/stderr contract	Bounded Docker journal/log files and experiment receipts
Health	Local probe executable	Compose health status and host monitoring
Receiver activation	Fail-closed check	Per-identity activation marker, created only during cutover
Backups	Restore-compatible formats	Encrypted or access-controlled snapshots and checksums

GitHub CLI authentication and unrelated local developer tooling are explicitly outside the migration critical path and may be re-established separately by the operator.

4 Credential and state categories

Only categories are listed here. No values, fragments, endpoints containing secrets, or private-key material belong in this document.

- VM SSH access and verified host-key records.
- Per-VM inference API authorization and matching inference user mapping.
- One Telegram bot token per identity.
- Internal OpenClaw gateway authorization.
- Model-provider and Codex authentication profiles.
- Optional connector OAuth credentials used by a particular identity.
- Telegram allowlists, chat routing rules, and identity policy files.
- Persistent state roots, ownership, filesystem modes, and backup location.
- Optional GitHub/gh authentication, handled outside this migration.

The migration copies existing required secrets through bounded, non-logging mechanisms. Operators manually re-establish only credentials that are host-bound, expired, unavailable for bounded transfer, or intentionally kept outside the copied state. Credential rotation remains post-migration cleanup by current owner decision and is not a cutover gate unless authentication fails or active misuse is observed.

5 Implementation phases

Phase 0: Freeze and reconcile

1. Re-audit the four live laptop supervisors and prove exactly one receiver per identity.
2. Freeze image inputs: repository commits, reviewed dirty-file hashes, dependency lockfiles, base image digest, and wrapper checksums.
3. Reconcile the current identity delta manifest against every container mount. Classify each path as immutable image content, read-only config, read-write state, secret, ephemeral cache, or excluded material.
4. Record disk requirements, identity-to-VM assignment, and rollback owner.

Gate 0: no unclassified production path; no secret in the build context; all pins and expected hashes recorded.

Phase 1: Build reproducible image and Compose stack

1. Create a minimal Dockerfile with a digest-pinned supported base image. Install dependencies from pinned manifests, copy only allowlisted source, and run as a non-root user where feasible.
2. Add an entrypoint that validates identity, required mounts, file modes, state schema, and activation marker before starting. It must exit nonzero on an unknown identity or incomplete configuration.
3. Define four Compose services across two VM-specific overlays. Use read-only root filesystems where compatible, temporary filesystems for scratch, bounded resource/log settings, and no broad host-directory mounts.
4. Add health checks that verify process liveness and local readiness without calling Telegram or consuming updates.
5. Generate a software bill of materials or at minimum an image digest, package inventory, source pins, and build transcript.

Gate 1: clean local build; image contains no recognizable secret material; provider-free test suite passes inside the image; Compose config renders successfully; all services remain polling-disabled.

Phase 2: Install Docker and stage offline

1. Verify official package provenance and install Docker Engine plus Compose on both existing VMs. No new paid resources are created.
2. Create fixed host directories for images/manifests, per-identity config, secrets, state, backup snapshots, and receipts. Apply least-privilege modes.
3. Transfer the image by a checksummed archive or pull it from an approved registry by immutable digest. Transfer Compose manifests separately.
4. Start every service in offline/no-poll mode with placeholder or omitted Telegram authorization. Run compilation, dependency, state-schema, provider, and health checks that cannot poll Telegram.

Gate 2: both VMs reproduce hashes; all services become healthy in offline mode; zero target pollers are observed; laptop pollers remain exactly one per identity.

Phase 3: Stage private state and rehearse rollback

1. Stop each offline service before state import. Copy preliminary state and required secrets into only that identity's mounts using a bounded allowlist.
2. Verify file counts, hashes where stable, modes, owners, state schema, and absence of cross-identity files. Do not print content.
3. Start again with polling disabled. Exercise provider authorization and local response generation without Telegram update consumption where possible.
4. Rehearse container stop, state backup, state restore into a disposable location, Compose restart, and laptop supervisor restart commands.

Gate 3: private mounts validate; offline provider smoke passes; backup/restore rehearsal passes; rollback commands and estimated time are recorded.

Phase 4: Per-identity production cutover

Use the same transaction for ProtoCosmo2, ProtoCosmo, Protomega, and Protomega2, one at a time. ProtoCosmo2 remains the first candidate because its preliminary state is already staged, but the operator may reorder lanes if its runtime acceptance prerequisites are incomplete.

1. Announce the lane and create a pre-cutover laptop state snapshot plus receiver-count receipt.

2. Stop the identity through its owning laptop supervisor. Verify the process is absent and cannot auto-restart. Leave all other identities running.
3. Copy the final mutable delta, including cursor and pending durable work, to that identity's target mounts. Verify metadata and state consistency.
4. Create only that identity's activation marker and start its Compose service. Confirm one target receiver and zero laptop receivers for the token.
5. Send fresh human-authored Telegram canaries: direct/group text as applicable, reply routing, and a safe attachment. Confirm exactly one response, correct identity, correct destination, preserved state, and no stale replay.
6. Restart the container and repeat a fresh canary. Inspect bounded logs for duplicate polling, authentication errors, dropped work, or cross-routing.
7. Accept the lane, or roll it back immediately before touching the next.

Gate 4 per identity: exactly one receiver; all fresh canaries and restart recovery pass; mutable state advances on the target; rollback remains available.

Phase 5: Host restart, reboot, and soak

1. After all four lanes pass independently, restart each Compose stack under systemd and verify identity isolation and receiver counts.
2. Reboot one VM at a time. Keep the other VM and laptop rollback artifacts intact. Confirm Docker, systemd, mounts, and services recover in correct order.
3. Repeat fresh text and attachment canaries after each reboot.
4. Soak for an agreed interval while monitoring process health, restart counts, disk growth, scheduled work, provider failures, routing, and duplicate receiver symptoms.

Gate 5: both VMs survive reboot; all four identities pass post-reboot canaries; soak shows no unresolved severity-one or severity-two defects.

Phase 6: Closeout and deferred cleanup

1. Freeze final image digest, Compose files, mount manifest, backup/restore runbook, test receipts, and identity assignment in the project record.
2. Keep laptop services stopped but restartable until the soak owner accepts the migration. Archive rather than delete rollback state.
3. Re-establish optional host tooling, including GitHub/gh, outside the receiver cutover path.
4. Perform deferred credential rotation and revoke superseded access after migration acceptance. Update secret mounts one identity at a time and retest.
5. Remove the persistent migration cron worker only after all acceptance gates and the durable record are complete.

6 Rollback transaction

Rollback is per identity unless a shared host failure requires reverting both identities assigned to that VM.

1. Stop the target Compose service and remove its activation marker.
2. Verify no target receiver remains and prevent automatic restart.
3. Preserve failed-target logs and a checksummed state snapshot for diagnosis. Do not overwrite the pre-cutover laptop snapshot.
4. If safe and schema-compatible, copy only target state advances needed to avoid lost durable work; otherwise restore the recorded laptop pre-cutover state and explicitly record the possible replay window.

5. Start the owning laptop supervisor. Confirm exactly one laptop receiver.
6. Send a fresh canary and verify identity, routing, and state continuity.
7. Record the failure, evidence, and the gate that must pass before retry.

Rollback triggers include duplicate receivers, wrong-chat routing, failed fresh canaries, cursor regression, repeated crash/restart, mount/permission errors, unbounded disk growth, or inability to prove receiver exclusivity.

7 Acceptance matrix

Area	Required evidence	Pass condition
Build	Base digest, image digest, source pins, dependency inventory, clean test log	Rebuild/test succeeds; no secrets detected
Isolation	Compose render, mount inventory, permission audit	No broad or cross-identity mutable mounts
Offline stage	Per-VM hashes, health status, process audit	Healthy services; zero VM pollers
Cutover	Before/after receiver counts and supervisor receipts	Exactly one receiver for each identity
Function	Fresh text, reply-routing, and attachment canaries	Correct single response in correct chat
State	Cursor/session/task continuity receipts	No regression, loss, or unexpected replay
Recovery	Container restart, stack restart, VM reboot receipts	Automatic recovery with fresh canary pass
Rollback	Rehearsal and retained laptop snapshot	Documented command path is usable within the cutover window
Soak	Health, restart, storage, schedule, and routing observations	No open high-severity defect
Closeout	Updated project, experiment, task, decision, and daily memory records	Evidence is durable and migration worker can retire

8 Operational sequence and ownership

The migration worker performs one bounded, checkpointed action per cycle. It must finish within its execution budget, record evidence before the checkpoint deadline, and never turn an ambiguous state into a cutover. Human operators provide fresh canary messages and resolve external-account credentials when a bounded transfer is not appropriate. Ben and Elija have equivalent authority for IT and permissioning decisions, subject to identity verification and normal platform safety constraints.

Status reports to Telegram should be short and limited to verified milestones, actual blockers, a cutover warning, a rollback event, or completion. They must never include secret values or fragments.

9 Immediate next actions

1. Freeze the Docker/Compose specification and reconcile it against the existing identity delta manifest.
2. Build the credential-free image locally from the already verified source pins; run provider-free tests inside it and scan the build context/image for secret material.
3. Install Docker/Compose on VM1 and VM2 from reviewed official packages, then transfer the image and manifests by checksum.
4. Pass offline Compose health and state-mount smokes on both targets.
5. Resume the first ProtoCosmo2 lane only after its runtime, mount, and rollback gates pass; then proceed one identity at a time.

10 Decision summary

The containerized approach is adopted because it reduces host drift, makes runtime pins portable, gives repeatable supervision and health checks, and simplifies future VM moves. It does not remove the distributed-systems risks at the Telegram boundary. Single-receiver exclusivity, state transactionality, fresh external canaries, reboot validation, and laptop rollback therefore remain the decisive acceptance conditions.